

# Solution Homework 2

Spring 2025

## Exercise 1 (10 pt), Forces, discussion questions, test your intuition

These questions expect not only an answer, but an explanation of your reasoning.

*To receive full credit, these answers should include both the underlying physics that explains your answer, but how you feel about that answer (i.e. are you confident? do you like this answer? do it unsettle you? it's ok to feel uncomfortable right now with these ideas; physics intuition is developed and often has to be resolved with our everyday experiences).*

- 1a (2pt) Single force. Can an object affected only by a single force have zero acceleration?

## Exercise 2 (10 pt), setting up forces, Newton's second law

Useful material here to read is

1. Taylor chapters 1.3 and 1.4 and
2. Malthe-Sørenssen chapters 5.1, 5.2 and 5.3

A person jumps from an airplane, falling freely for several seconds before the person pulls the cord of the parachute and the parachute unfolds.

- 2a (3pt) Identify the forces acting on the parachuter and draw a free-body diagram of the parachuter before the person has pulled the cord. Include a brief discussion of any assumptions you make, motivate and justify your choices.

## Exercise 3 (10 pt), Space shuttle with air resistance

Useful material here to read is

1. Malthe-Sørenssen chapters 5.1, 5.2 and 5.3

During lift-off of the space shuttle the engines provide a force of  $35 \times 10^6$  N. The mass of the shuttle is approximately  $2 \times 10^6$  kg.

- 3a (3pt) Draw a free-body diagram of the space shuttle immediately after lift-off.

## Exercise 4 (15 pt), now hitting a golf ball

Useful material here to read is

1. Taylor chapters 1.3-1.6 and
2. Malthe-Sørenssen chapter 6.3-6.4 and 7.1-7.3

**Taylor exercise 1.35.** The formulae you obtain here will be useful for the numerical exercises below (see exercise 6 below).

## Exercise 5 (15 pt), ball thrown along a sloped ramp

**Taylor exercise 1.39.** Make sure to draw your setup clearly, show your free-body diagram, and explain any assumptions you make to solve the problem.

*Repeated below consistent with [fair use practices](#)*

A ball is thrown with initial speed  $v_0$  up an inclined plane. The plane is inclined at an angle  $\phi$  above the horizontal, and the ball's initial velocity is at an angle  $\theta$  above the plane. Choose axes with  $x$  measured up the slope,  $y$  normal to the slope, and  $z$  across it.

- 5a (5 pt) Write down Newton's second law using these axes and find the ball's position as a function of time. **Make sure to include the FBD and any assumptions you make.**

We can now find the range of the ball by finding the time at which it hits the ground. We set  $y(t) = 0$  and solve for the nonzero  $t$ . We can use  $y(t) = 0$  because we have chosen the origin to be at the initial location of the ball.

$$0 = v_0 \sin \theta t - \frac{1}{2} g \cos \phi t^2$$

$$0 = t(v_0 \sin \theta - \frac{1}{2} g \cos \phi t)$$

And thus,

$$t_{flight} = \frac{2v_0 \sin \theta}{g \cos \phi}$$

We use that time to find the range of the ball, which is the  $x$ -coordinate at that time.

$$x(t_{flight}) = v_0 \cos \theta t_{flight} - \frac{1}{2} g \sin \phi t_{flight}^2$$

$$x(t_{flight}) = v_0 \cos \theta \frac{2v_0 \sin \theta}{g \cos \phi} - \frac{1}{2} g \sin \phi \left( \frac{2v_0 \sin \theta}{g \cos \phi} \right)^2$$

for which we can use this handy identity:

$$\cos \phi \cos \theta - \sin \phi \sin \theta = \cos(\phi + \theta)$$

This will allow us to simplify the expression for the range.

$$x(t_{flight}) = \frac{2v_0^2 \sin \theta \cos \theta}{g \cos \phi} - \frac{2v_0^2 \sin^2 \theta \sin \phi}{g \cos^2 \phi}$$

$$x(t_{flight}) = \frac{2v_0^2 \sin \theta}{g} \left( \frac{\cos \phi \cos \theta - \sin \phi \sin \theta}{\cos^2 \phi} \right)$$

$$R(v_0, \theta, \phi) = x(t_{flight}) = \frac{2v_0^2 \sin \theta}{g} \frac{\cos(\phi + \theta)}{\cos^2 \phi}$$

## Exercise 6 (40pt), Numerical elements, moving to more than one dimension

**This exercise should be handed in as a jupyter-notebook** at D2L. Remember to write your name(s).

Last week we:

1. Analytically mapped 1D motion over some time
2. Gained practice with functions
3. Reviewed vectors and matrices in Python

This week we will:

1. Practice using Python syntax and variable manipulation
2. Utilize analytical solutions to create more refined functions
3. Work in two, three or even higher dimensions

This material will then serve as background for the numerical part of homework 3. The first part is a simple warm-up, with hints and suggestions you can use for the code to write below.

```
In [1]: %matplotlib inline

# As usual, here are some useful packages we will be using. Feel free to use more and experiment as you wish.

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits import import mplot3d
%matplotlib inline
```

In class (the falling baseball example) we used an analytical expression for the height of a falling ball. In the first homework we used instead the position from experiment (Usain Bolt's 100m record run) and stored this information with one-dimensional arrays in Python.

Let us get some practice with this. The cell below creates two arrays, one containing the times to be analyzed and the other containing the  $x$  and  $y$  components of the position vector at each point in time. This is a two-dimensional object. The second array is initially empty. Then we define the initial position to be  $x = 2$  and  $y = 1$ . Take a look at the code and comments to get an understanding of what is happening. Feel free to play around with it.

```
In [2]: tf = 4 #length of value to be analyzed
dt = .001 # step sizes
t = np.arange(0.0,tf,dt) # Creates an evenly spaced time array going from 0 to 3.999, with step sizes .001
```

```
p = np.zeros((len(t), 2)) # Creates an empty array of [x,y] arrays (our vectors). Array size is same as the
p[0] = [2.0,1.0] # This sets the initial position to be x = 2 and y = 1
```

Below we are printing specific values of our array to see what is being stored where. The first number in the array  $r[]$  represents which array iteration we are looking at, while the number after the represents which listed number in the array iteration we are getting back.

```
In [3]: print(p[0]) # Prints the first array
        print(p[0,:]) # Same as above, these commands are interchangeable
```

```
[2. 1.]
[2. 1.]
```

```
In [4]: print(p[3999]) # Prints the 4000th array
```

```
[0. 0.]
```

```
In [5]: print(p[0,0]) # Prints the first value of the first array
```

```
2.0
```

```
In [6]: print(p[0,1]) # Prints the second value of first array
        print(p[:,0]) # Prints the first value of all the arrays
```

```
1.0
```

```
[2. 0. 0. ... 0. 0. 0.]
```

Then try running this cell. Notice how it gives an error since we did not implement a third dimension into our arrays

```
In [7]: print(p[:,2])
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[7], line 1
----> 1 print(p[:,2])

IndexError: index 2 is out of bounds for axis 1 with size 2
```

In the cell below we want to manipulate the arrays. In this example we make each vector's  $x$  component valued the same as their respective vector's position in the iteration and the  $y$  value will be twice that value, except for the first vector, which we have already set. That is we have  $p[0] = [2, 1], p[1] = [1, 2], p[2] = [2, 4], p[3] = [3, 6], \dots$

Here we set up an array for  $x$  and  $y$  values.

```
In [ ]: for i in range(1,3999):
        p[i] = [i,2*i]
        # Checker cell to make sure your code is performing correctly
        c = 0
        for i in range(0,3999):
            if i == 0:
                if p[i,0] != 2.0:
                    c += 1
                if p[i,1] != 1.0:
                    c += 1
            else:
                if p[i,0] != 1.0*i:
                    c += 1
                if p[i,1] != 2.0*i:
                    c += 1

        if c == 0:
            print("Success!")
        else:
            print("There is an error in your code")
```

Success!

You could also think of an alternative way of storing the above information. Feel free to explore how to store multidimensional objects.

Last week we studied Usain Bolt's 100m run and in class we studied a falling baseball. We made basic plots of the baseball moving in one dimension. This week we will be working with a three-dimensional variant. This will be useful for our next homeworks and numerical projects.

Assume we have a soccer ball moving in three dimensions with the following trajectory:

$$1. x(t) = 10t \cos 45^\circ$$

$$2. y(t) = 10t \sin 45^\circ$$

$$3. z(t) = 10t - \frac{9.81}{2}t^2$$

Now let us create a three-dimensional (3D) plot using these equations. In the cell below we write the equations into their respective labels. We fix a final time in the code below.

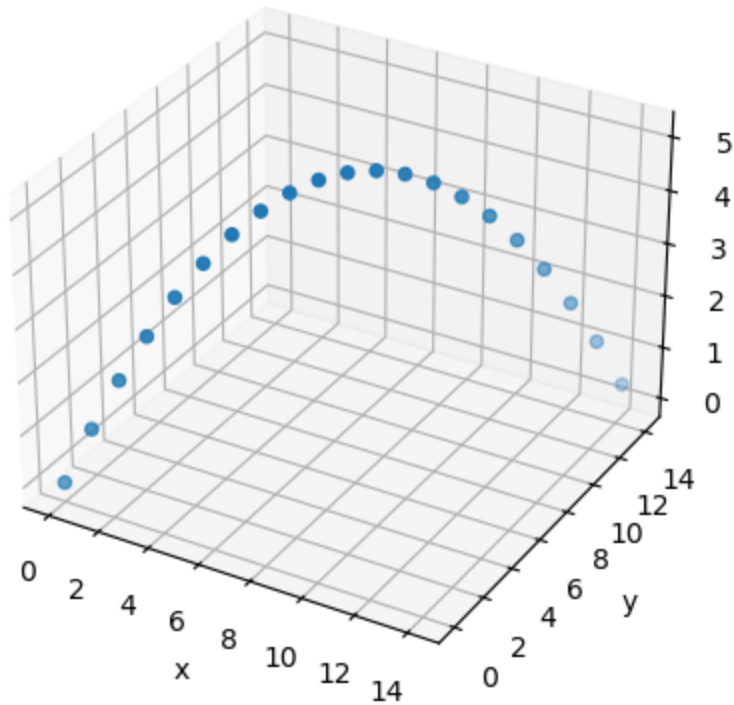
Important Concept: Numpy comes with many mathematical packages, some of them being the trigonometric functions sine, cosine, tangent. We are going to utilize these this week. Additionally, these functions work with radians, so we will also be using a function from Numpy that converts degrees to radians.

```
In [ ]: tf = 2.04 # The final time to be evaluated
dt = 0.1 # The time step size
t = np.arange(0,tf,dt) # The time array
theta_deg = 45 # Degrees
theta_rad = np.radians(theta_deg) # Converts degrees to their radian counterparts
x = 10*t*np.cos(theta_rad) # Equation for our x component, utilizing np.cos() and our calculated radians
y = 10*t*np.sin(theta_rad) # Put the y equation here
z = 10*t-9.81/2*t**2 # Put the z equation here
```

Then we plot it

```
In [ ]: ## Once you have entered the proper equations in the cell above, run this cell to plot in 3D
fig = plt.axes(projection='3d')
fig.set_xlabel('x')
fig.set_ylabel('y')
fig.set_zlabel('z')
fig.scatter(x,y,z)
```

```
Out[ ]: <mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x1245623f0>
```



- 6a (8pt) How would you express  $x(t)$ ,  $y(t)$ ,  $z(t)$  for this problem as a single vector,  $\mathbf{r}(t)$ ?

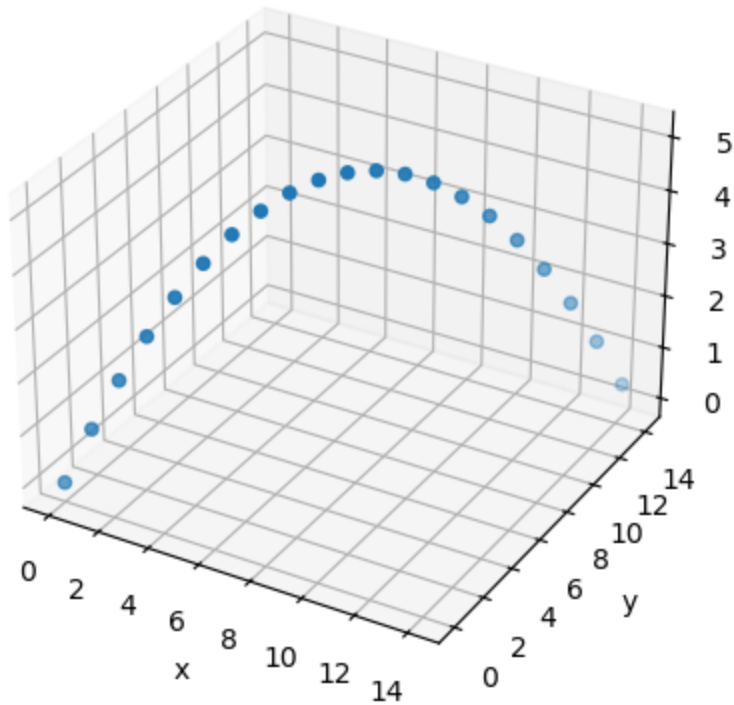
Then run the code and plot using the array  $\mathbf{r}$

```
In [ ]: ## Solution
r = np.array((10*t*np.cos(theta_rad), 10*t*np.sin(theta_rad), 10*t - 9.81/2*t**2))

## Run this code to plot using our r array
fig = plt.axes(projection='3d')
fig.set_xlabel('x')
fig.set_ylabel('y')
fig.set_zlabel('z')
fig.scatter(r[0],r[1],r[2])
```

```
Out[ ]: <mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x124553200>
```





- 6b (8pt) What do you think the benefits and/or disadvantages are from expressing our three equations as a single array/vector? This can be both from a computational and physics stand point. Use the **Numpy** package to also print the maximum  $x$ ,  $y$  and  $z$  components from  $\mathbf{r}$ .

Complete Exercise 4 above (Taylor exercise 1.35) before moving further. (Recall that the golf ball was hit due east at an angle  $\theta$  with respect to the horizontal, and the coordinate directions are  $x$  measured east,  $y$  north, and  $z$  vertically up.)

- 6c (8pt) What is the analytical solution for our theoretical golf ball's position  $\mathbf{r}(t)$  over time from Exercise 4? Also what is the formula for the time  $t_f$  when the golf ball hits the ground? Use this to develop a program with a function called for example Golfball that utilizes our analytical solutions. This program should take in an initial velocity and the angle  $\theta$  that the golfball was hit with in degrees. It should also produce a 3D graph of the motion. You need also to find the maximum values for  $x$ ,  $y$  and  $z$ .
- 6d (8pt) Given initial values of  $v_i = 90\text{m/s}$ ,  $\theta = 30^\circ$ , what would our maximum  $x$ ,  $y$  and  $z$  components be?

- 6e (8pt) Given initial values of  $v_i = 45\text{m/s}$ ,  $\theta = 45^\circ$ , what would our maximum x, y and z components be?

```
In [ ]: def golfball(vi, theta):
    theta_rad = np.radians(theta)
    tf = vi*np.sin(theta_rad)/4.9
    dt = 0.1
    t = np.arange(0,tf,dt)
    x = t*vi*np.cos(theta_rad)
    y = 0*t
    z = t*vi*np.sin(theta_rad)-9.81/2*t**2

    r = [x,y,z]
    fig = plt.axes(projection='3d')
    fig.set_xlabel('x')
    fig.set_ylabel('y')
    fig.set_zlabel('z')
    fig.scatter(r[0],r[1],r[2])

    print("Maximum x:", tf*vi*np.cos(theta_rad))
    print("Maximum y:", 0)
    print("Maximum z:", (tf/2.0)*vi*np.sin(theta_rad)-9.81/2*(tf/2.0)**2)

print("6d:")
golfball(90, 30)
print("6e:")
golfball(45, 45)
```

6d:

Maximum x: 715.7965072095869

Maximum y: 0

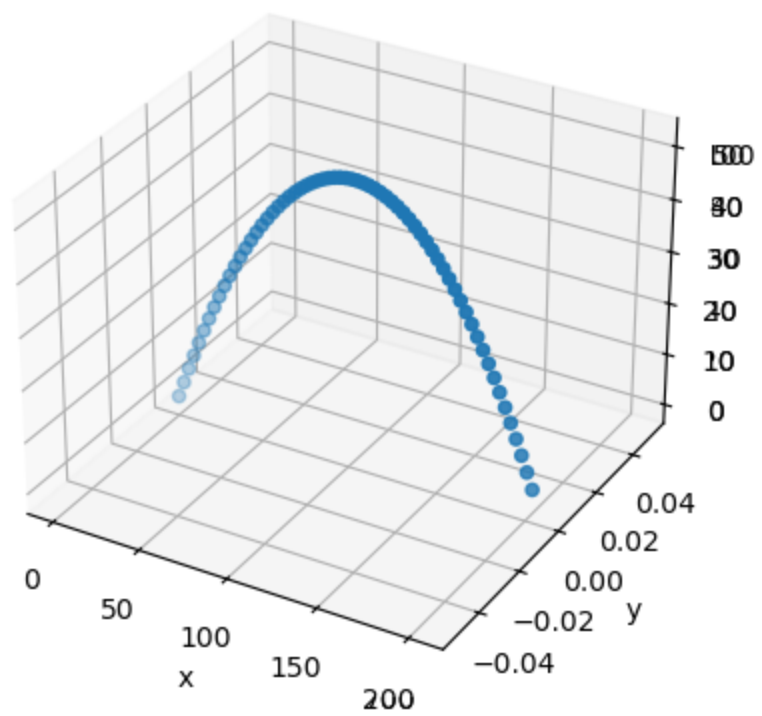
Maximum z: 103.21090170762179

6e:

Maximum x: 206.63265306122446

Maximum y: 0

Maximum z: 51.60545085381089



In [ ]: